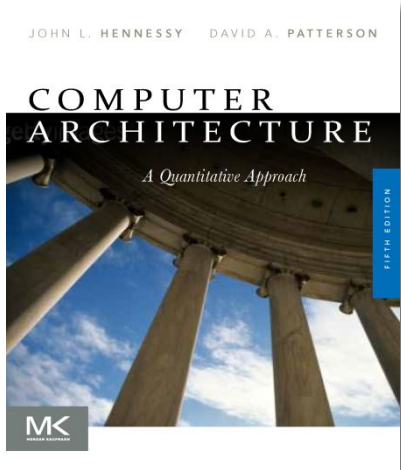




Unit – 1 Contd...

Instruction level parallelism

By

Dr. Sandhya . S

Dept. of CSE, RVCE

Slide Sources: Based on CA: aQA by Hennessy/Patterson. Supplemented from various freely downloadable sources

Introduction:

- The value of the CPI (cycles per instruction) for a pipelined processor,-

Pipeline CPI = Ideal pipeline CPI + Structural stalls + Data hazard stalls + Control stalls

Instruction-Level Parallelism (ILP)

- **Basic Block (BB) ILP is quite small**
 - BB: a straight-line code sequence with no branches in except to the entry and no branches out except at the exit
 - average dynamic branch frequency 15% to 25%
=> 4 to 7 instructions execute between a pair of branches
 - Plus instructions in BB likely to depend on each other
- **To obtain substantial performance enhancements, we must exploit ILP across multiple basic blocks**
- **Simplest: loop-level parallelism to exploit parallelism among iterations of a loop. E.g.,**

```
for (i=1; i<=1000; i=i+1)  
    x[i] = x[i] + y[i];
```

Loop-Level Parallelism

- Exploit loop-level parallelism to parallelism by “unrolling loop” either by
 1. dynamic via branch prediction or
 2. static via loop unrolling by compiler(Another way is vectors, to be covered later)
- Determining instruction dependence is critical to Loop Level Parallelism
- If 2 instructions are
 - parallel, they can execute simultaneously in a pipeline of arbitrary depth without causing any stalls (assuming no structural hazards)
 - dependent, they are not parallel and must be executed in order, although they may often be partially overlapped

Data Dependence and Hazards

- Instr_j is **data dependent (true dependence)** on Instr_i :
 1. Instruction i produces a result that may be used by instruction j .

 I: add **r1**, r2, r3
J: sub r4, **r1**, r3

2. Instruction j is data dependent on instruction k , and instruction k is data dependent on instruction i .
- If two instructions are data dependent, they cannot execute simultaneously or be completely overlapped
 - Data dependence in instruction sequence $\Rightarrow$ data dependence in source code $\Rightarrow$ effect of original data dependence must be preserved
 - If data dependence caused a hazard in pipeline, called a **Read After Write (RAW) hazard**

Determine Data Dependency?

```
Loop:      L.D      F0,0(R1)
           ADD.D    F4,F0,F2
           S.D      F4,0(R1)
           DADDUI   R1,R1,#-8
           BNE      R1,R2,LOOP
```

ILP and Data Dependencies, Hazards

- HW/SW must preserve **program order**:
order instructions would execute in if executed sequentially as determined by original source program
 - Dependences are a property of **programs**
- Presence of dependence indicates **potential** for a hazard, but actual hazard and length of any stall is property of the **pipeline organization**.
- Importance of the data dependencies
 - 1) indicates the possibility of a hazard
 - 2) determines order in which results must be calculated
 - 3) sets an upper bound on how much parallelism can possibly be exploited
- HW/SW goal: exploit parallelism by preserving program order **only where it affects the outcome of the program**

Name Dependence #1: Anti-dependence

- **Name dependence**: when 2 instructions use same register or memory location, called a **name**, but no flow of data between the instructions associated with that name; **2 versions of name dependence**
- Instr_j writes operand before Instr_i reads it

 I: sub r4, **r1**, r3
J: add **r1**, r2, r3
K: mul r6, r1, r7

Called an “**anti-dependence**” by compiler writers.
This results from reuse of the name “**r1**”

- If anti-dependence caused a hazard in the pipeline, called a **Write After Read (WAR) hazard**

Name Dependence #2: Output dependence

- Instr_j writes operand before Instr_i writes it.

 I: sub r1, r4, r3
J: add r1, r2, r3
K: mul r6, r1, r7

- Called an “**output dependence**” by compiler writers
This also results from the reuse of name “r1”
- If output-dependence caused a hazard in the pipeline, called a **Write After Write (WAW) hazard**
- Instructions involved in a name dependence can execute simultaneously **if name used in instructions is changed** so instructions do not conflict
 - **Register renaming** resolves name dependence for regs
 - Either by compiler or by HW

Data Hazards

The possible data hazards are,-

- 1. *RAW (read after write)***
- 2. *WAW (write after write)***
- 3. *WAR (write after read)***

RAR (read after read) case is not a hazard.

Control Dependencies

- Every instruction is control dependent on some set of branches, and, in general, these control dependencies must be preserved to preserve program order

```
if p1 {  
    S1;  
};  
if p2 {  
    S2;  
}
```

- **S1 is control dependent on p1**, and **S2 is control dependent on p2** but not on p1.

Control Dependence Ignored

- **Control dependence need not be preserved**
 - willing to execute instructions that should not have been executed, thereby violating the control dependences, **if** can do so without affecting correctness of the program
- **Instead, 2 properties critical to program correctness are**
 - 1) **exception behavior** and
 - 2) **data flow**

Exception Behavior

- Preserving exception behavior
⇒ any changes in instruction execution order must not change how exceptions are raised in program
(⇒ no new exceptions)

- Example:

```
DADDU      R2 , R3 , R4
BEQZ      R2 , L1
LW        R1 , 0 (R2)
L1 :
```

– (Assume branches not delayed)

- Problem with moving LW before BEQZ?

Data Flow

- **Data flow**: actual flow of data values among instructions that produce results and those that consume them
 - branches make flow dynamic, determine which instruction is supplier of data
- **Example:**
DADDU R1, R2, R3
BEQZ R4, L
DSUBU R1, R5, R6
L: ...
OR R7, R1, R8
- OR depends on DADDU or DSUBU?
Must preserve data flow on execution

Software Techniques - Example

- This code, add a scalar to a vector:

```
for (i=1000; i>0; i=i-1)
    x[i] = x[i] + s;
```
- Assume following latencies for all examples
 - Ignore delayed branch in these examples

<i>Instruction producing result</i>	<i>Instruction using result</i>	<i>Latency in cycles</i>	<i>stalls between in cycles</i>
FP ALU op	Another FP ALU op	4	3
FP ALU op	Store double	3	2
Load double	FP ALU op	1	1
Load double	Store double	1	0
Integer op	Integer op	1	0

FP Loop: Where are the Hazards?

- First translate into MIPS code:
 - To simplify, assume 8 is lowest address

```
Loop: L.DF0,0(R1) ;F0=vector element
      ADD.D F4,F0,F2 ;add scalar from F2
      S.DF4, 0(R1); ;store result
      DADDUIR1,R1,-8 ;decrement pointer 8B (DW)
      BNEZ R1,Loop ;branch R1!=zero
```

Show how this loop on MIPS, both scheduled and unscheduled including stalls or idle clock cycles.

FP Loop Showing Stalls

```
1 Loop:  L.D F0,0(R1)    ;F0=vector element
2  stall
3  ADD.D  F4,F0,F2      ;add scalar in F2
4  stall
5  stall
6  S.D 0(R1),F4        ;store result
7  DADDUI R1,R1,-8      ;decrement pointer 8B (DW)
8  stall              ;assumes can't forward to branch
9  BNEZ   R1,Loop      ;branch R1!=zero
```

*Instruction Latency in
producing result using result clock cycles*

FP ALU op Another FP ALU op 3

FP ALU op Store double 2

Load doubleFP ALU op 1

- **9 clock cycles: Rewrite code to minimize stalls?**

Revised FP Loop Minimizing Stalls

```
1 Loop:  L.D F0 , 0 (R1)
2  DADDUI R1 , R1 , -8
3  ADD.D  F4 , F0 , F2
4  stall
5  stall
6  S.D 8 (R1) , F4 ;altered offset when move DSUBUI
7  BNEZ    R1 , Loop
```

Swap DADDUI and S.D by changing address of S.D

*Instruction Instruction Latency in
producing result using result clock cycles*

FP ALU op Another FP ALU op 3

FP ALU op Store double 2

Load double FP ALU op 1

7 clock cycles, but just 3 for execution (L.D, ADD.D, S.D), 4 for loop overhead; How make faster?

Copyright © 2012, Elsevier Inc.

All rights reserved. Dept. of

CSE, RVCE

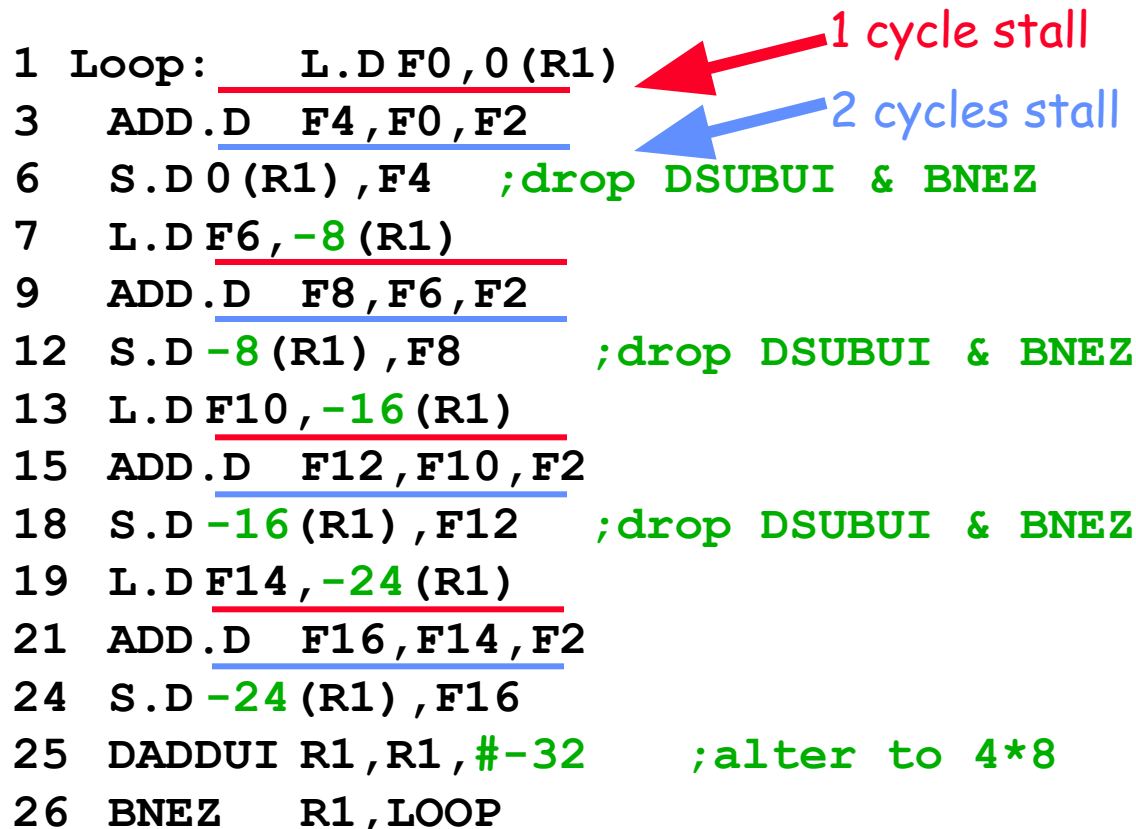
Loop Unrolling

Show the loop unrolled so that there are four copies of the loop body. Eliminate any obviously redundant computations and do not reuse any of the registers.

Show the unrolled loop in the previous example after it has been scheduled for the pipeline with the given latencies earlier.

Unroll Loop Four Times (straightforward way)

```
1 Loop:  L.D F0, 0(R1)
3  ADD.D F4, F0, F2
6  S.D 0(R1), F4 ;drop DSUBUI & BNEZ
7  L.D F6, -8(R1)
9  ADD.D F8, F6, F2
12 S.D -8(R1), F8 ;drop DSUBUI & BNEZ
13 L.D F10, -16(R1)
15 ADD.D F12, F10, F2
18 S.D -16(R1), F12 ;drop DSUBUI & BNEZ
19 L.D F14, -24(R1)
21 ADD.D F16, F14, F2
24 S.D -24(R1), F16
25 DADDUI R1, R1, #-32 ;alter to 4*8
26 BNEZ R1, LOOP
```



**Rewrite loop to
minimize stalls?**

27 clock cycles, or 6.75 per iteration
(Assumes R1 is multiple of 4)

Unrolled Loop Detail

- Do not usually know upper bound of loop
- Suppose it is n , and we would like to unroll the loop to make k copies of the body
- Instead of a single unrolled loop, we generate a pair of consecutive loops:
 - 1st executes $(n \bmod k)$ times and has a body that is the original loop
 - 2nd is the unrolled body surrounded by an outer loop that iterates (n/k) times
- For large values of n , most of the execution time will be spent in the unrolled loop

5 Loop Unrolling Decisions

- Requires understanding how one instruction depends on another and how the instructions can be changed or reordered given the dependences:
1. Determine loop unrolling useful by finding that loop iterations were independent (except for maintenance code)
 2. Use different registers to avoid unnecessary constraints forced by using same registers for different computations
 3. Eliminate the extra test and branch instructions and adjust the loop termination and iteration code
 4. Determine that loads and stores in unrolled loop can be interchanged by observing that loads and stores from different iterations are independent
 - Transformation requires analyzing memory addresses and finding that they do not refer to the same address
 5. Schedule the code, preserving any dependences needed to yield the same result as the original code

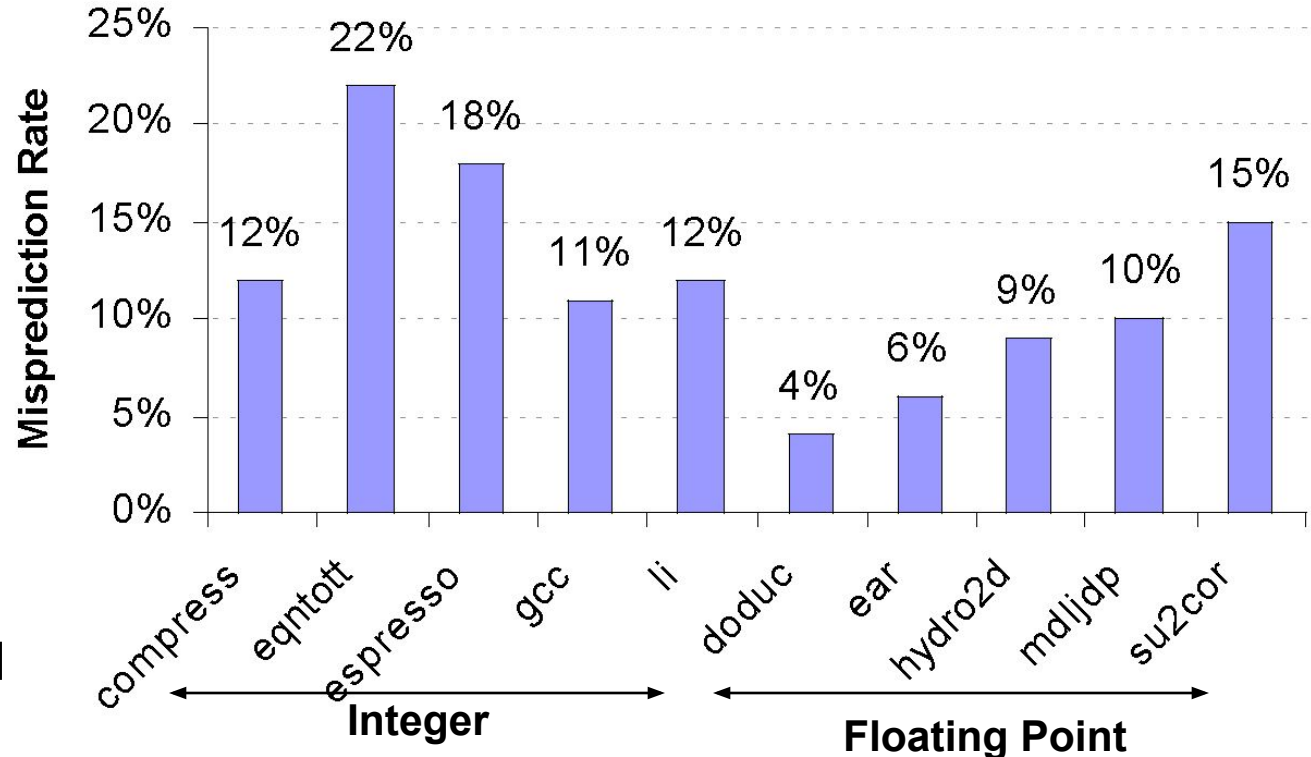
3 Limits to Loop Unrolling

1. **Decrease in amount of overhead amortized with each extra unrolling**
 - Amdahl's Law
2. **Growth in code size**
 - For larger loops, concern it increases the instruction cache miss rate
3. **Register pressure: potential shortfall in registers created by aggressive unrolling and scheduling**
 - If not be possible to allocate all live values to registers, may lose some or all of its advantage
 - Loop unrolling reduces impact of branches on pipeline; another way is **branch prediction**

Static Branch Prediction

- To reorder code around branches, need to predict branch statically when compile
- Simplest scheme is to predict a branch as taken
 - Average misprediction = untaken branch frequency = 34% SPEC

- More accurate scheme predicts branches using profile information collected from earlier runs, and modify prediction based on last run:



Dynamic Branch Prediction

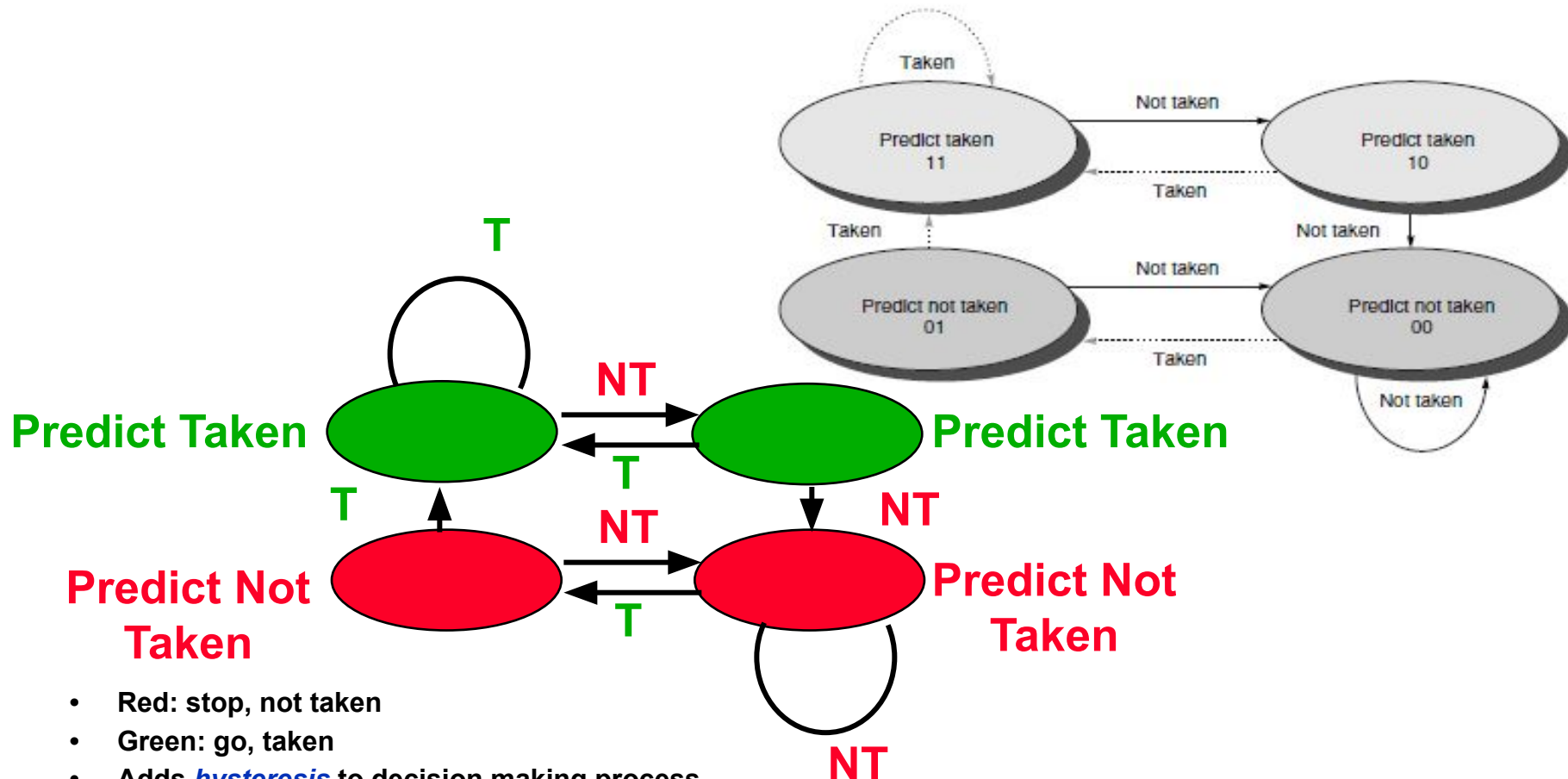
- **Why does prediction work?**
 - Underlying algorithm has regularities
 - Data that is being operated on has regularities
 - Instruction sequence has redundancies that are artifacts of way that humans/compiler think about problems
- **Is dynamic branch prediction better than static branch prediction?**
 - Seems to be
 - There are a small number of important branches in programs which have dynamic behavior

Dynamic Branch Prediction

- Performance = $f(\text{accuracy, cost of misprediction})$
- Branch History Table: Lower bits of PC address index table of 1-bit values
 - Says whether or not branch taken last time
- Problem: in a loop, 1-bit BHT will cause two mispredictions:
 - End of loop case, when it exits instead of looping as before
 - First time through loop on *next* time through code, when it predicts exit instead of looping

Dynamic Branch Prediction

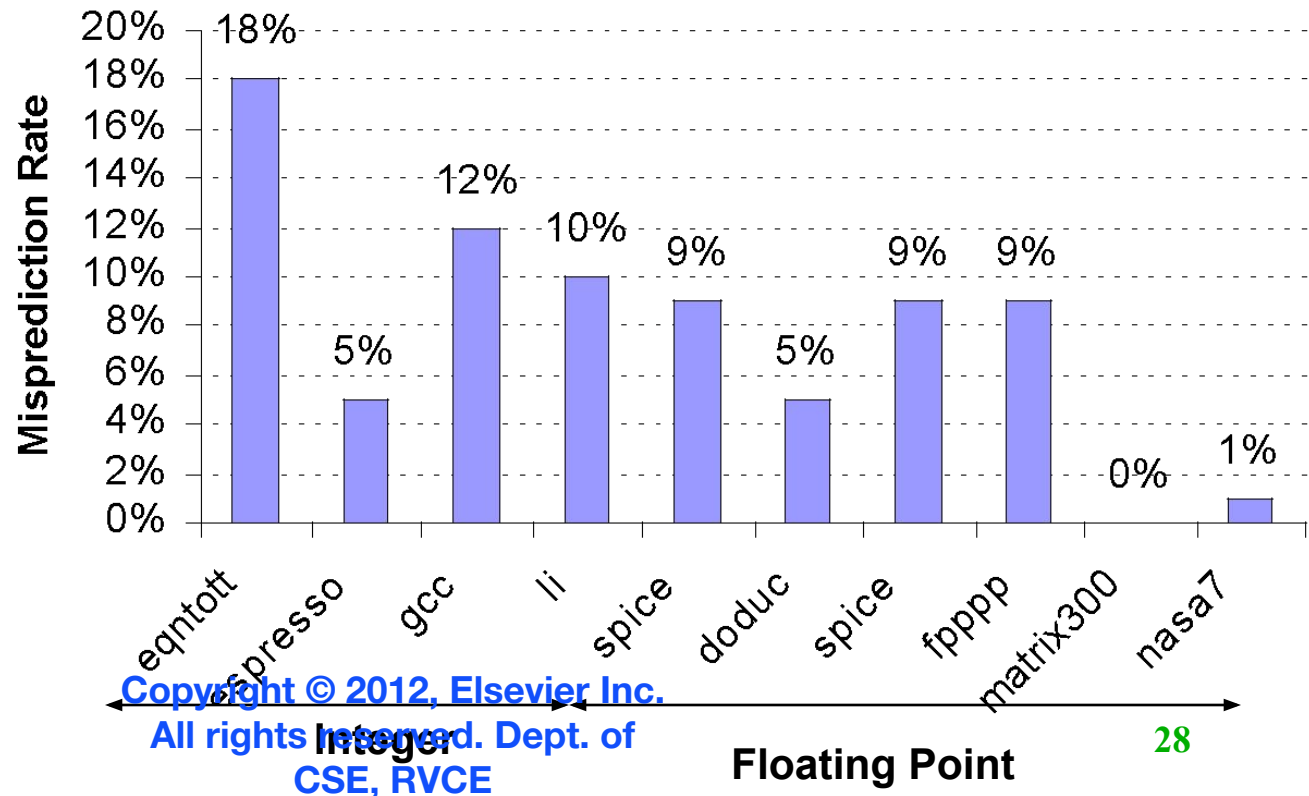
- Solution: 2-bit scheme where change prediction only if get misprediction *twice*



- Red: stop, not taken
- Green: go, taken
- Adds *hysteresis* to decision making process

BHT Accuracy

- **Mispredict because either:**
 - Wrong guess for that branch
 - Got branch history of wrong branch when index the table
- **4096 entry table:**



Correlated Branch Prediction

```
if (aa==2)
aa=0;
if (bb==2)
bb=0;
if (aa!=bb) {
```

A (1,2) predictor uses the behavior of the last branch to choose from among a pair of 2-bit branch predictors in predicting a particular branch.

- MIPS Code:

```

                DADDIU    R3,R1,#-2
                BNEZ      R3,L1          ;branch b1    (aa!=2)
                DADD      R1,R0,R0      ;aa=0
L1:             DADDIU    R3,R2,#-2
                BNEZ      R3,L2          ;branch b2    (bb!=2)
                DADD      R2,R0,R0      ;bb=0
L2:             DSUBU     R3,R1,R2       ;R3=aa-bb
                BEQZ      R3,L3          ;branch b3    (aa==bb)
```

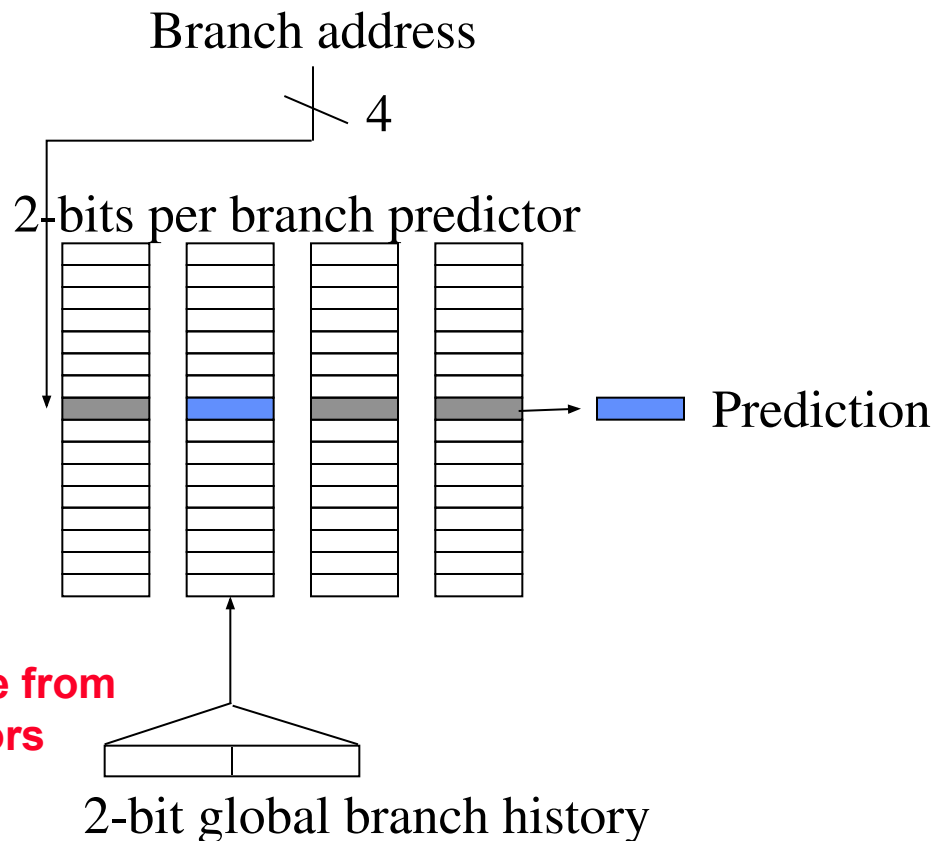
Correlated Branch Prediction

- Idea: record m most recently executed branches as taken or not taken, and use that pattern to select the proper n -bit branch history table
- In general, (m,n) predictor means record last m branches to select between 2^m history tables, each with n -bit predictor.
 - Thus, old 2-bit BHT is a $(0,2)$ predictor
- Global Branch History: m -bit shift register keeping T/NT status of last m branches.
- Each entry in table has m n -bit predictors.

Correlating Branches

(2,2) predictor

- Behavior of recent branches selects between four predictions of next branch, updating just that prediction



A (1,2) predictor uses the behavior of the last branch to choose from among a pair of 2-bit branch predictors in predicting a particular branch.

Correlating Branches

- The number of bits in an (m,n) predictor is

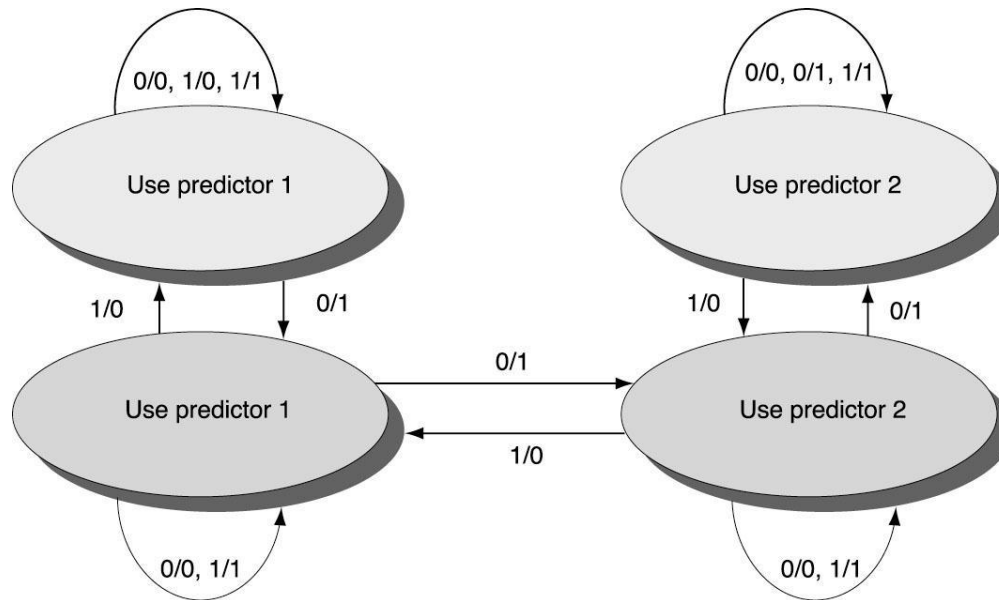
$2^m \times n \times \text{Number of prediction entries selected by the branch address}$

A 2-bit predictor with no global history is simply a $(0,2)$ predictor

Example: How many bits are in the $(0,2)$ branch predictor with 4K entries? How many entries are in a $(2,2)$ predictor with the same number of bits?

Tournament Predictors

- Multilevel branch predictor
- Use n -bit saturating counter to choose between predictors
- Usual choice between global and local predictors

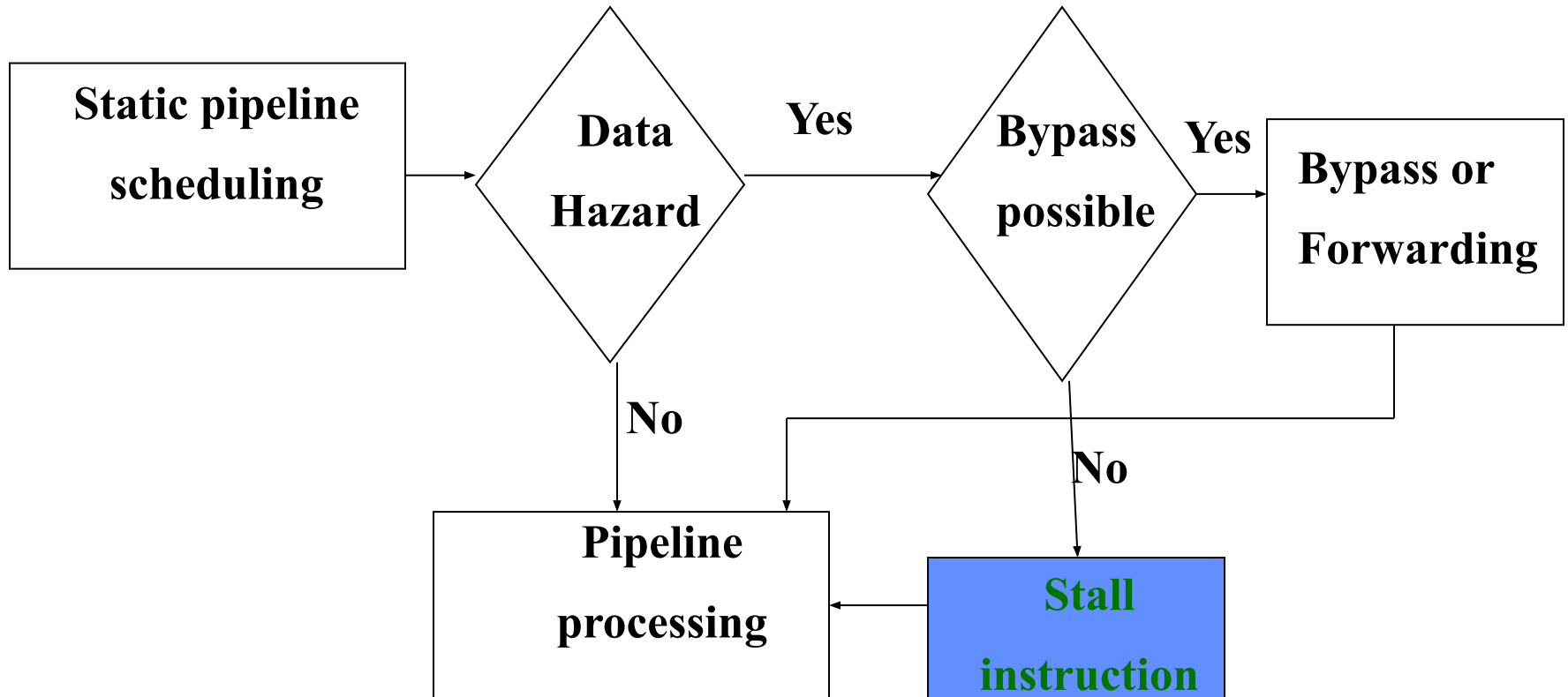


Tournament Predictors

Tournament predictor using, say, 4K 2-bit counters indexed by local branch address. Chooses between:

- **Global predictor**
 - 4K entries index by history of last 12 branches ($2^{12} = 4K$)
 - Each entry is a standard 2-bit predictor
- **Local predictor**
 - Local history table: 1024 10-bit entries recording last 10 branches, index by branch address
 - The pattern of the last 10 occurrences of that particular branch used to index table of 1K entries with 3-bit saturating counters

Why Dynamic Scheduling...?



Dynamic scheduling reduces this stall
via ILP: Instruction Level Parallelism

Goal of ILP: To get as many instructions as possible
executing

Dynamic Scheduling: Key Ideas

- Old paradigm (classic MIPS 5-stage integer pipeline):
 - *in-order instruction issue and execution*
 - can cause *unnecessary delay* of instructions that also *wastes hardware resources* by keeping them idle through the delay
 - e.g.,

DIV.D F0, F2, F4

ADD.D F6, F0, F8 # ADD.D and S.D are stalled by

S.D F6, 0(R1) # true data dependences

SUB.D F8, F10, F14 # SUB.D and MUL.D are ready to execute

MUL.D F6, F10, F8 # but blocked by previous stalls!

Dynamic Scheduling: Key Ideas

- New paradigm:

- *in-order issue* but allow *out-of-order execution* (i.e., ILP = parallel execution of instructions) and, therefore, *out-of-order completion*

- e.g.,

DIV.D F0, F2, F4

ADD.D F6, F0, F8

S.D F6, 0(R1)

SUB.D F8, F10, F14

MUL.D F6, F10, F8

- *without waiting* for ADD.D and S.D to complete execution try to execute SUB.D and MUL.D
- this out-of-order execution raises two *potential hazards* that do not exist in the classic pipeline with in-order execution
 - » WAR hazard: the antidependence between ADD.D and SUB.D
 - » WAW hazard: the output dependence between ADD.D and MUL.D

Dynamic Scheduling: Key Ideas

- solution: eliminate WAR and WAW hazards by *register renaming*

- e.g.,

DIV.D F0, F2, F4

ADD.D F6, F0, F8

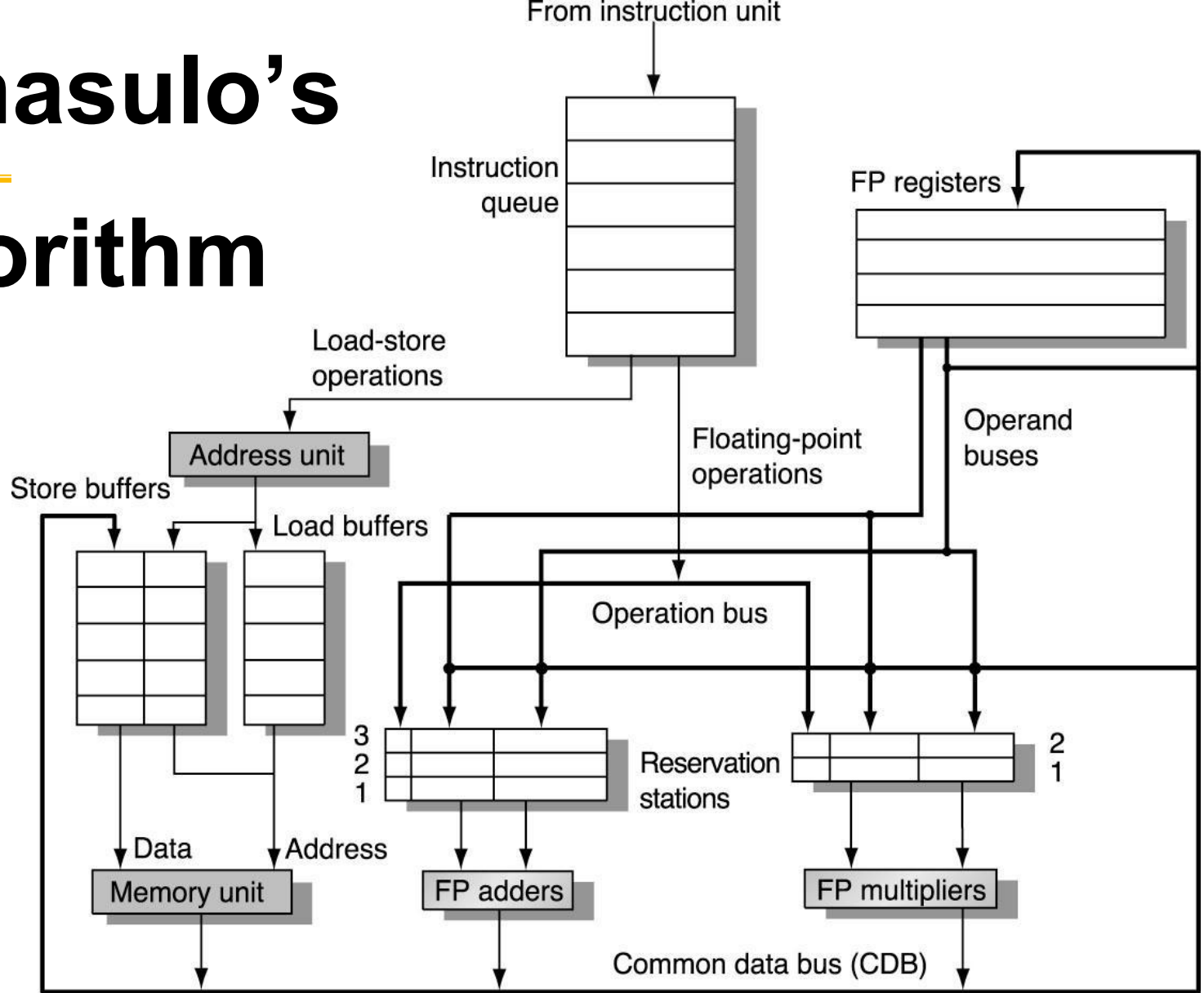
S.D F6, 0(R1)

SUB.D F8, F10, F14

MUL.D F6, F10, F8

- Tomasulo provides register renaming via reservation stations
 - » reservation stations *fetch and buffer* an operand as soon as it is available, eliminating need to go to register to get operand
 - » pending instructions designate reservation stations that will provide input values
 - » results are passed directly from *functional units* where they are computed to the reservation stations where they are required over the *common data bus (CDB)* – *bypassing* registers

Tomasulo's Algorithm



Note: reservations stations do *not* form a queue! They all have independent access to FP op unit
Note: there may be multiple or pipelined FP op units – conceptually same!

Basic structure of MIPS floating-point unit based on Tomasulo

Copyright © 2012, Elsevier Inc.

All rights reserved. Dept. of

© 2005 Elsevier Science (USA). All rights reserved.

Tomasulo's Algorithm: Three Stages

1. **Issue**: get instruction from Instruction Queue
 - if reservation station free (no structural hazard), control issues instruction to reservation station, and sends to reservation station operand values (or reservation station source for values)

2. **Execution**: operate on operands (EX)
 - when both operands ready then execute;
if not ready, watch CDB for result

3. **Write result**: finish execution (WB)
 - write on CDB to all awaiting units;
mark reservation station available

Tomasulo's Algorithm:

Data Structures

Reservation station fields

- **Op:** Operation to be performed on source operands S1 and S2
- **Qj, Qk:** The reservation stations that will produce the corresponding operand; value of 0 indicates source operand is *already available* in Vj or Vk, or is unnecessary
- **Vj, Vk:** The value of the source operands. Only one of the V or Q fields is valid for each operand. For loads, Vk field holds offset
- **A:** Holds information for the memory address calculation for load and store. Initially, immediate field of instruction is stored here; after address calculation, effective address is stored
- **Busy:** Reservation station and related functional unit occupied

Register file field

- **Qi:** Number of the reservation station that contains the operation whose results will be stored into this register; value of 0 (or blank) indicates value is register contents, i.e., no instruction targets this register

Examples

1. L.D F6, 34 (R2)
2. L.D F2, 45 (R3)
3. MUL.D F0, F2, F4
4. SUB.D F8, F2, F6
5. DIV.D F10, F0, F6
6. ADD.D F6, F8, F2

We run Tomasulo's algorithm on the above code sequence in three different examples:

- A. Data structures when the only the first load has completed*
- B. Data structures when MUL.D is about to write*
- C. Data structures cycle by cycle*

Example A1: Instructions

All instructions have issued but only the first L.D has completed and written its result

Instruction	Instruction Status		
	Issue	Execute	Write Results
L.D F6, 34(R2)	X	X	X
L.D F2, 45(R3)	X	X	
MUL.D F0, F2, F4	X		
SUB.D F8, F2, F6	X		
DIV.D F10, F0, F6	X		
ADD.D F6, F8, F2	X		

Example A1: Reservation Stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	yes	LOAD					45 + Regs[R3]
Add1	yes	SUB		Mem[34+Regs[R2]]	Load2		
Add2	yes	ADD			Add1	Load2	
Add3	no						
Mult1	yes	MUL		Regs[F4]	Load2		
Mult2	yes	DIV		Mem[34+Regs[R2]]	Mult1		

Addi indicates *i*th reservation station for the FP add unit, etc.

Example A1: Registers

Field	F0	F2	F4	F6	F8	F10	F12.....F30
Qi	Mult1	Load2		Add2	Add1	Mult2	

Floating point registers

Latencies

- Assume operation latencies
 - load: 2 clock cycles
 - add/sub: 2 clock cycles
 - multiply: 10 clock cycles
 - divide: 40 clock cycles

1. L.D	F6, 34 (R2)
2. L.D	F2, 45 (R3)
3. MUL.D	F0, F2, F4
4. SUB.D	F8, F2, F6
5. DIV.D	F10, F0, F6
6. ADD.D	F6, F8, F2

Example B: Instructions

Instruction	Instruction Status		
	Issue	Execute	Write Results
L.D F6, 34(R2)	X	X	X
L.D F2, 45(R3)	X	X	X
MUL.D F0, F2, F4	X	X	
SUB.D F8, F2, F6	X	X	X
DIV.D F10, F0, F6	X		
ADD.D F6, F8, F2	X	X	X

When MUL.D is about to write

Example B: Reservation Stations

Name	Busy	Op	Vj	Vk	Qj	Qk	A
Load1	no						
Load2	no						
Add1	no						
Add2	no						
Add3	no						
Mult1	yes	MUL	Mem[45+Regs[R3]]	Regs[F4]			
Mult2	yes	DIV		Mem[34+Regs[R2]]	Mult1		

Addi indicates *i*th reservation station for the FP add unit, etc.

Example B: Registers

Field	F0	F2	F4	F6	F8	F10	F12.....F30
Qi	Mult1					Mult2	

Floating point registers

Tomasulo Loop Example

```
Loop: LD      F0      0      R1
      MULTD    F4      F0      F2
      SD      F4      0      R1
      SUBI     R1      R1      #8
      BNEZ     R1      Loop
```

- Assume multiply takes 4 clocks
- Assume first load takes 8 clocks (cache miss?), second load takes 4 clocks (hit)
- To be clear, will show clocks for SUBI, BNEZ
- Reality: integer instructions ahead

Loop Example Cycle 0

Instruction status				Execution Write			Busy	Address			
Instruction	j	k	iteration	Issue	complete	Result					
LD F0	0	R1	1			Load1	No	Qi			
MULT F4	F0	F2	1			Load2	No				
SD F4	0	R1	1			Load3	No				
LD F0	0	R1	2			Store1	No				
MULT F4	F0	F2	2			Store2	No				
SD F4	0	R1	2			Store3	No				
Reservation Stations				S1	S2	RS for j	RS for k	Code:			
Time	Name	Busy	Op	Vj	Vk	Qj	Qk				
0	Add1	No									
0	Add2	No									
0	Add3	No									
0	Mult1	No									
0	Mult2	No									
Register result status											
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
0	80	Qi									

Loop Example Cycle 1

Instruction status				Execution Write			Busy	Address
Instruction	j	k	iteration	Issue	complete	Result		
LD F0	0	R1	1	1		Load1	Yes	80
MULT F4	F0	F2	1			Load2	No	Qi
SD F4	0	R1	1			Load3	No	
LD F0	0	R1	2			Store1	No	
MULT F4	F0	F2	2			Store2	No	
SD F4	0	R1	2			Store3	No	
Reservation Stations				S1	S2	RS for j	RS for k	Code:
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	
0	Add1	No						
0	Add2	No						
0	Add3	No						
0	Mult1	No						
0	Mult2	No						
0	Add1	No						LD F0 0 R1
0	Add2	No						MULT F4 F0 F2
0	Add3	No						SD F4 0 R1
0	Mult1	No						SUBI R1 R1 #8
0	Mult2	No						BNEZ R1 Loop
Register result status								
Clock	R1	F0	F2	F4	F6	F8	F10	F12 ... F30
1	80 Qi	Load1						

Loop Example Cycle 2

Instruction status				Execution Write			Busy	Address
Instruction	j	k	iteration	Issue	complete	Result		
LD F0	0	R1	1	1		Load1	Yes	80
MULT F4	F0	F2	1	2		Load2	No	
SD F4	0	R1	1			Load3	No	Qi
LD F0	0	R1	2			Store1	No	
MULT F4	F0	F2	2			Store2	No	
SD F4	0	R1	2			Store3	No	

Reservation Stations			S1	S2	RS for j	RS for k	Code:
Time	Name	Busy Op	Vj	Vk	Qj	Qk	
0	Add1	No					LD F0 0 R1
0	Add2	No					MULT F4 F0 F2
0	Add3	No					SD F4 0 R1
0	Mult1	Yes					SUBI R1 R1 #8
0	Mult2	No					BNEZ R1 Loop

Register result status		F0	F2	F4	F6	F8	F10	F12	...	F30
Clock	R1									
2	80 Qi	Load1		Mult1						

Loop Example Cycle 3

Instruction status				Execution Write			Busy	Address
Instruction	j	k	iteration	Issue	complete	Result		
LD F0	0	R1	1	1		Load1	Yes	80
MULT F4	F0	F2	1	2		Load2	No	
SD F4	0	R1	1	3		Load3	No	Qi
LD F0	0	R1	2			Store1	Yes	80
MULT F4	F0	F2	2			Store2	No	Mult1
SD F4	0	R1	2			Store3	No	

Reservation Stations				S1	S2	RS for j	RS for k	Code:
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	
0	Add1	No						LD F0 0 R1
0	Add2	No						MULT F4 F0 F2
0	Add3	No						SD F4 0 R1
0	Mult1	Yes	MULTD		R(F2)	Load1		SUBI R1 R1 #8
0	Mult2	No						BNEZ R1 Loop

Register result status				F0	F2	F4	F6	F8	F10	F12	...	F30
Clock	R1											
3	80	Qi		Load1		Mult1						

- **Note: MULT1 has no registers names in RS**
- Copyright © 2012, Elsevier Inc.
All rights reserved. Dept. of
CSE, RVCE

Loop Example Cycle 4

Instruction status				Execution Write			Busy	Address			
Instruction	j	k	iteration	Issue	complete	Result					
LD F0	0	R1	1	1		Load1	Yes	80			
MULT F4	F0	F2	1	2		Load2	No				
SD F4	0	R1	1	3		Load3	No	Qi			
LD F0	0	R1	2			Store1	Yes	80			
MULT F4	F0	F2	2			Store2	No				
SD F4	0	R1	2			Store3	No				
Reservation Stations				S1	S2	RS for j	RS for k	Code:			
Time	Name	Busy	Op	Vj	Vk	Qj	Qk				
0	Add1	No						LD F0 0 R1			
0	Add2	No						MULT F4 F0 F2			
0	Add3	No						SD F4 0 R1			
0	Mult1	Yes	MULTD		R(F2)	Load1		SUBI R1 R1 #8			
0	Mult2	No						BNEZ R1 Loop			
Register result status											
Clock	R1	F0	F2	F4	F6	F8	F10	F12 ...	F30		
4	72	Qi	Load1	Mult1							

Loop Example Cycle 5

Instruction status				Execution Write			Busy	Address
Instruction	j	k	iteration	Issue	complete	Result		
LD F0	0	R1	1	1		Load1	Yes	80
MULT F4	F0	F2	1	2		Load2	No	
SD F4	0	R1	1	3		Load3	No	Qi
LD F0	0	R1	2			Store1	Yes	80
MULT F4	F0	F2	2			Store2	No	
SD F4	0	R1	2			Store3	No	
Reservation Stations				S1	S2	RS for j	RS for k	Code:
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	
0	Add1	No						LD F0 0 R1
0	Add2	No						MULT F4 F0 F2
0	Add3	No						SD F4 0 R1
0	Mult1	Yes	MULTD		R(F2)	Load1		SUBI R1 R1 #8
0	Mult2	No						BNEZ R1 Loop
Register result status								
Clock	R1	F0	F2	F4	F6	F8	F10	F12 ... F30
5	72 Qi	Load1		Mult1				

Loop Example Cycle 6

Instruction status					Execution	Write						
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1			Load1	Yes	80			
MULT F4	F0	F2	1	2			Load2	Yes	72			
SD F4	0	R1	1	3			Load3	No		Qi		
LD F0	0	R1	2	6			Store1	Yes	80	Mult1		
MULT F4	F0	F2	2				Store2	No				
SD F4	0	R1	2				Store3	No				
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	Time	Name	Busy	Op	V _j	V _k	Q _j	Q _k	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	Yes	MULTD		R(F2)	Load1		SUBI	R1	R1	#8
	0	Mult2	No						BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
6	72	Qi	Load2		Mult1							

- **Note: F0 never sees Load1 result**
- Copyright © 2012, Elsevier Inc.
All rights reserved. Dept. of
CSE, RVCE

Loop Example Cycle 7

Instruction status					Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address		
LD F0	0	R1	1	1			Load1	Yes	80		
MULT F4	F0	F2	1	2			Load2	Yes	72		
SD F4	0	R1	1	3			Load3	No		Qi	
LD F0	0	R1	2	6			Store1	Yes	80	Mult1	
MULT F4	F0	F2	2	7			Store2	No			
SD F4	0	R1	2				Store3	No			
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>			
	Time	Name	Busy	Op	V _j	V _k	Q _j	Q _k	Code:		
	0	Add1	No						LD	F0	0 R1
	0	Add2	No						MULT	F4	F0 F2
	0	Add3	No						SD	F4	0 R1
	0	Mult1	Yes	MULTD		R(F2)	Load1		SUBI	R1	R1 #8
	0	Mult2	Yes	MULTD		R(F2)	Load2		BNEZ	R1	Loop
Register result status											
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30
7	72	Qi	Load2		Mult2						

- **Note: MULT2 has no registers names in RS**
- Copyright © 2012, Elsevier Inc.
All rights reserved. Dept. of
CSE, RVCE

Loop Example Cycle 8

Instruction status				Execution Write			Busy	Address
Instruction	j	k	iteration	Issue	complete	Result		
LD F0	0	R1	1	1		Load1	Yes	80
MULT F4	F0	F2	1	2		Load2	Yes	72
SD F4	0	R1	1	3		Load3	No	Qi
LD F0	0	R1	2	6		Store1	Yes	80
MULT F4	F0	F2	2	7		Store2	Yes	72
SD F4	0	R1	2	8		Store3	No	

Reservation Stations				S1	S2	RS for j	RS for k	Code:
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	
0	Add1	No						LD F0 0 R1
0	Add2	No						MULT F4 F0 F2
0	Add3	No						SD F4 0 R1
0	Mult1	Yes	MULTD		R(F2)	Load1		SUBI R1 R1 #8
0	Mult2	Yes	MULTD		R(F2)	Load2		BNEZ R1 Loop

Register result status				F0	F2	F4	F6	F8	F10	F12	...	F30
Clock	R1											
8	72	Qi		Load2		Mult2						

Loop Example Cycle 9

Instruction status				Execution Write			Busy	Address
Instruction	j	k	iteration	Issue	complete	Result		
LD F0	0	R1	1	1	9	Load1	Yes	80
MULT F4	F0	F2	1	2		Load2	Yes	72
SD F4	0	R1	1	3		Load3	No	Qi
LD F0	0	R1	2	6		Store1	Yes	80
MULT F4	F0	F2	2	7		Store2	Yes	72
SD F4	0	R1	2	8		Store3	No	

Reservation Stations				S1	S2	RS for j	RS for k	Code:
Time	Name	Busy	Op	Vj	Vk	Qj	Qk	
0	Add1	No						LD F0 0 R1
0	Add2	No						MULT F4 F0 F2
0	Add3	No						SD F4 0 R1
0	Mult1	Yes	MULTD		R(F2)	Load1		SUBI R1 R1 #8
0	Mult2	Yes	MULTD		R(F2)	Load2		BNEZ R1 Loop

Register result status				F0	F2	F4	F6	F8	F10	F12	...	F30
Clock	R1											
9	64	Qi		Load2		Mult2						

- Load1 completing; what is waiting for it?
- Copyright © 2012, Elsevier Inc.
All rights reserved. Dept. of
CSE, RVCE

Loop Example Cycle 10

Instruction status				Execution Write			Busy	Address		
Instruction	j	k	iteration	Issue	complete	Result				
LD F0	0	R1	1	1	9	10	Load1	No	Qi	
MULT F4	F0	F2	1	2			Load2	Yes		72
SD F4	0	R1	1	3			Load3	No		
LD F0	0	R1	2	6	10		Store1	Yes	80	Mult1
MULT F4	F0	F2	2	7			Store2	Yes	72	Mult2
SD F4	0	R1	2	8			Store3	No		
Reservation Stations				S1	S2	RS for j	RS for k	Code:		
Time	Name	Busy	Op	Vj	Vk	Qj	Qk			
0	Add1	No						LD	F0	0 R1
0	Add2	No						MULT	F4	F0 F2
0	Add3	No						SD	F4	0 R1
4	Mult1	Yes	MULTD	M(80)	R(F2)			SUBI	R1	R1 #8
0	Mult2	Yes	MULTD		R(F2)	Load2		BNEZ	R1	Loop
Register result status										
Clock	R1	F0	F2	F4	F6	F8	F10	F12	...	F30
10	64	Qi	Load2		Mult2					

- Load2 completing; what is waiting for it?
- Copyright © 2012, Elsevier Inc.
All rights reserved. Dept. of
CSE, RVCE

Loop Example Cycle 11

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2			Load2	No				
SD F4	0	R1	1	3			Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	Yes	80	Mult1		
MULT F4	F0	F2	2	7			Store2	Yes	72	Mult2		
SD F4	0	R1	2	8			Store3	No				
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>	<i>Code:</i>			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	3	Mult1	Yes	MULTD	M(80)	R(F2)			SUBI	R1	R1	#8
	4	Mult2	Yes	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
11	64	Qi	Load3		Mult2							

Loop Example Cycle 12

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2			Load2	No				
SD F4	0	R1	1	3			Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	Yes	80	Mult1		
MULT F4	F0	F2	2	7			Store2	Yes	72	Mult2		
SD F4	0	R1	2	8			Store3	No				
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	Time	Name	Busy	Op	V _j	V _k	Q _j	Q _k	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	2	Mult1	Yes	MULTD	M(80)	R(F2)			SUBI	R1	R1	#8
	3	Mult2	Yes	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
12	64	Qi	Load3		Mult2							

Loop Example Cycle 13

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2			Load2	No				
SD F4	0	R1	1	3			Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	Yes	80	Mult1		
MULT F4	F0	F2	2	7			Store2	Yes	72	Mult2		
SD F4	0	R1	2	8			Store3	No				
Reservation Stations					S1	S2	RS for j	RS for k				
	Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	1	Mult1	Yes	MULTD	M(80)	R(F2)			SUBI	R1	R1	#8
	2	Mult2	Yes	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
13	64	Qi	Load3		Mult2							

Loop Example Cycle 14

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2	14		Load2	No				
SD F4	0	R1	1	3			Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	Yes	80	Mult1		
MULT F4	F0	F2	2	7			Store2	Yes	72	Mult2		
SD F4	0	R1	2	8			Store3	No				
Reservation Stations					S1	S2	RS for j	RS for k				
	Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	Yes	MULTD	M(80)	R(F2)			SUBI	R1	R1	#8
	1	Mult2	Yes	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
14	64	Qi	Load3		Mult2							

- Mult1 completing; what is waiting for it?
- Copyright © 2012, Elsevier Inc.
All rights reserved. Dept. of
CSE, RVCE

Loop Example Cycle 15

Instruction status					Execution		Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2	14	15	Load2	No				
SD F4	0	R1	1	3			Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	Yes	80	M(80)*R(F		
MULT F4	F0	F2	2	7	15		Store2	Yes	72	Mult2		
SD F4	0	R1	2	8			Store3	No				
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	Time	Name	Busy	Op	V _j	V _k	Q _j	Q _k	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	No						SUBI	R1	R1	#8
	0	Mult2	Yes	MULTD	M(72)	R(F2)			BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
15	64	Qi	Load3		Mult2							

- Mult2 completing; what is waiting for it?

Loop Example Cycle 16

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2	14	15	Load2	No				
SD F4	0	R1	1	3			Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	Yes	80	M(80)*R(F		
MULT F4	F0	F2	2	7	15	16	Store2	Yes	72	M(72)*R(7		
SD F4	0	R1	2	8			Store3	No				
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	Time	Name	Busy	Op	V _j	V _k	Q _j	Q _k	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	Yes	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	No						BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
16	64	Qi	Load3		Mult1							

Loop Example Cycle 17

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2	14	15	Load2	No				
SD F4	0	R1	1	3			Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	Yes	80	M(80)*R(F		
MULT F4	F0	F2	2	7	15	16	Store2	Yes	72	M(72)*R(7		
SD F4	0	R1	2	8			Store3	Yes	64	Mult1		
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	Time	Name	Busy	Op	V _j	V _k	Q _j	Q _k	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	Yes	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	No						BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
17	64	Qi	Load3		Mult1							

Loop Example Cycle 18

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2	14	15	Load2	No				
SD F4	0	R1	1	3	18		Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	Yes	80	M(80)*R(F		
MULT F4	F0	F2	2	7	15	16	Store2	Yes	72	M(72)*R(7		
SD F4	0	R1	2	8			Store3	Yes	64	Mult1		
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	Time	Name	Busy	Op	Vj	Vk	Qj	Qk	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	Yes	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	No						BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
18	56	Qi	Load3		Mult1							

Loop Example Cycle 19

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2	14	15	Load2	No				
SD F4	0	R1	1	3	18	19	Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	No				
MULT F4	F0	F2	2	7	15	16	Store2	Yes	72	M(72)*R(7		
SD F4	0	R1	2	8			Store3	Yes	64	Mult1		
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	Time	Name	Busy	Op	V _j	V _k	Q _j	Q _k	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	Yes	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	No						BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
19	56	Qi	Load3		Mult1							

Loop Example Cycle 20

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2	14	15	Load2	No				
SD F4	0	R1	1	3	18	19	Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	No				
MULT F4	F0	F2	2	7	15	16	Store2	Yes	72	M(72)*R(7		
SD F4	0	R1	2	8	20		Store3	Yes	64	Mult1		
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	Time	Name	Busy	Op	V _j	V _k	Q _j	Q _k	Code:			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	Yes	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	No						BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
20	56	Qi	Load3		Mult1							

Loop Example Cycle 21

Instruction status						Execution	Write					
Instruction	<i>j</i>	<i>k</i>	<i>iteration</i>	<i>Issue</i>	<i>complete</i>	<i>Result</i>		Busy	Address			
LD F0	0	R1	1	1	9	10	Load1	No				
MULT F4	F0	F2	1	2	14	15	Load2	No				
SD F4	0	R1	1	3	18	19	Load3	Yes	64	Qi		
LD F0	0	R1	2	6	10	11	Store1	No				
MULT F4	F0	F2	2	7	15	16	Store2	No				
SD F4	0	R1	2	8	20	21	Store3	Yes	64	Mult1		
Reservation Stations					S1	S2	RS for <i>j</i>	RS for <i>k</i>				
	<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>	<i>Code:</i>			
	0	Add1	No						LD	F0	0	R1
	0	Add2	No						MULT	F4	F0	F2
	0	Add3	No						SD	F4	0	R1
	0	Mult1	Yes	MULTD		R(F2)	Load3		SUBI	R1	R1	#8
	0	Mult2	No						BNEZ	R1	Loop	
Register result status												
Clock	R1		F0	F2	F4	F6	F8	F10	F12	...	F30	
21	56	Qi	Load3		Mult1							

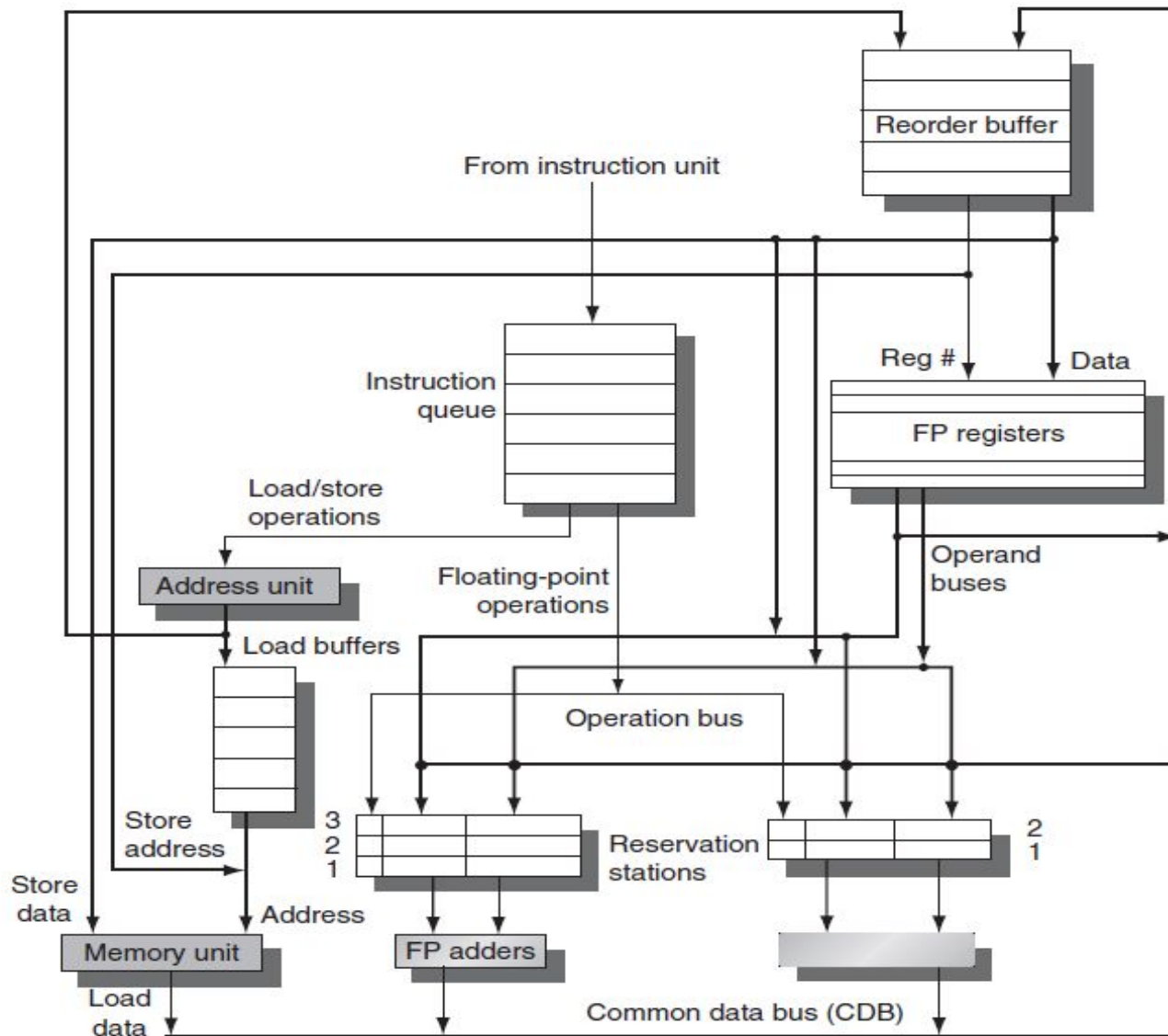
Steps in tomasulo algorithm

Instruction state	Wait until	Action or bookkeeping
Issue FP operation	Station r empty	if (RegisterStat[rs].Qi ≠ 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; if (RegisterStat[rt].Qi ≠ 0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0}; RS[r].Busy ← yes; RegisterStat[rd].Q ← r;
Load or store	Buffer r empty	if (RegisterStat[rs].Qi ≠ 0) {RS[r].Qj ← RegisterStat[rs].Qi} else {RS[r].Vj ← Regs[rs]; RS[r].Qj ← 0}; RS[r].A ← imm; RS[r].Busy ← yes;
Load only		RegisterStat[rt].Qi ← r;
Store only		if (RegisterStat[rt].Qi ≠ 0) {RS[r].Qk ← RegisterStat[rt].Qi} else {RS[r].Vk ← Regs[rt]; RS[r].Qk ← 0};
Execute FP operation	(RS[r].Qj = 0) and (RS[r].Qk = 0)	Compute result: operands are in Vj and Vk
Load/store step 1	RS[r].Qj = 0 & r is head of load-store queue	RS[r].A ← RS[r].Vj + RS[r].A;
Load step 2	Load step 1 complete	Read from Mem[RS[r].A]
Write result FP operation or load	Execution complete at r & CDB available	$\forall x$ (if (RegisterStat[x].Qi = r) {Regs[x] ← result; RegisterStat[x].Qi ← 0}); $\forall x$ (if (RS[x].Qj = r) {RS[x].Vj ← result; RS[x].Qj ← 0}); $\forall x$ (if (RS[x].Qk = r) {RS[x].Vk ← result; RS[x].Qk ← 0}); RS[r].Busy ← no;
Store	Execution complete at r & RS[r].Qk = 0	Mem[RS[r].A] ← RS[r].Vk; RS[r].Busy ← no;

Tomasulo Summary

- **Advantages**
 - prevents registers from being the bottleneck
 - eliminates WAR, WAW hazards
 - allows loop unrolling in HW
 - common data bus (CDB) broadcasts results to multiple instructions
- **Disadvantages**
 - hardware complexity
 - performance limited by associative stores required from CDB to reservation stations
 - performance limited by CDB bandwidth (CDB = bottleneck)
- **Lasting Contributions**
 - dynamic scheduling
 - register renaming
 - load/store disambiguation
- **Original Tomasulo implementation was on IBM 360/91**
 - famous modern descendants: Pentiums, PowerPCs, MIPS R10000,...

Hardware-Based Speculation



ROB - Fields

Each entry in the ROB contains four fields:

- **The instruction type,**
- **The destination field,**
- **The value field, and**
- **The ready field.**

Hardware-Based Speculation

With speculation there are the four steps involved in instruction execution:

- 1. *Issue***
- 2. *Execute***
- 3. *Write result***
- 4. *Commit***

Hardware-Based Speculation

Example Assume the same latencies for the floating-point functional units as in earlier examples: add is 2 clock cycles, multiply is 6 clock cycles, and divide is 12 clock cycles. Using the code segment below, the same one we used to generate Figure 3.8, show what the status tables look like when the MUL.D is ready to go to commit.

L.D	F6, 32(R2)
L.D	F2, 44(R3)
MUL.D	F0, F2, F4
SUB.D	F8, F2, F6
DIV.D	F10, F0, F6
ADD.D	F6, F8, F2

Hardware-Based Speculation

- Solution:

Reorder buffer						
Entry	Busy	Instruction		State	Destination	Value
1	No	L.D	F6,32(R2)	Commit	F6	Mem[32 + Regs[R2]]
2	No	L.D	F2,44(R3)	Commit	F2	Mem[44 + Regs[R3]]
3	Yes	MUL.D	F0,F2,F4	Write result	F0	#2 × Regs[F4]
4	Yes	SUB.D	F8,F2,F6	Write result	F8	#2 − #1
5	Yes	DIV.D	F10,F0,F6	Execute	F10	
6	Yes	ADD.D	F6,F8,F2	Write result	F6	#4 + #2

Reservation stations								
Name	Busy	Op	Vj	Vk	Qj	Qk	Dest	A
Load1	No							
Load2	No							
Add1	No							
Add2	No							
Add3	No							
Mult1	No	MUL.D	Mem[44 + Regs[R3]]	Regs[F4]			#3	
Mult2	Yes	DIV.D		Mem[32 + Regs[R2]]	#3		#5	

FP register status										
Field	F0	F1	F2	F3	F4	F5	F6	F7	F8	F10
Reorder #	3						6		4	5
Busy	Yes	No	No	No	No	No	Yes	...	Yes	Yes